BANKING AND DIGITAL TRANSACTION MANAGEMENT SYSTEM

IBI Bank — International Bank of India

PROJECT-BASED LEARNING (PBL) REPORT

Course / Subject Code	BTY39105
Institution	Brainware University
Department / Programme	B.Tech Cse-CS & DS / B.Tech Cse-Cys
Team / Group	G7
Team Members	Sk Jishan Ahamed [BWU/BCC/25/032] Naim Hasan [BWU/BCC/25/094] Raj Sekh [BWU/BCC/25/033]
Faculty Guide	MR.Mainul Hasan Indrasis Das
Submission Date	29/10/2026

1. Abstract and Introduction

Abstract. The Banking and Digital Transaction Management System is a database-backed application developed under the IBI Bank project identity. Its intended scope includes customer and account management, transaction recording, fund transfers, beneficiary management, loan records, EMI payment information, authentication, and administrative workflows. A MySQL relational database stores related entities, while a Node.js and Express backend provides API endpoints used by the web interface.

The project demonstrates how relational database concepts can support a practical information system: structured tables, keys and relationships, retrieval queries, aggregate reports, joins, nested queries, views, indexes, stored procedures, a user-defined function, triggers, and transaction control. The final report should include actual screenshots and execution evidence from the version submitted.

1.1 Background

Banking applications manage data that is both connected and sensitive. Customer details are associated with accounts; accounts are associated with transactions; transfers involve sending and receiving accounts; and loans can have multiple EMI payment records. A relational model helps represent these connections explicitly and enables users to retrieve operational details and management summaries.

1.2 Problem Statement

Maintaining banking information in disconnected records can create duplication, inconsistent updates, and difficulty producing accurate reports. The project addresses this by organizing banking entities in a MySQL database and providing a web interface and backend APIs for common operations. The submitted implementation and its actual constraints should be used as the source of truth for all claims in this report.

1.3 Objectives

- Organize banking data in related relational tables.
- Provide role-aware authentication and access to application functions.
- Support account, beneficiary, transfer, loan, and EMI workflows.
- Demonstrate SQL retrieval, reporting, and advanced database objects.
- Document design decisions and collect evidence for testing.

2. Scope, Requirements, and Technology

2.1 Project Scope

The application scope covers customer profiles, user login identities, bank accounts, branches, account types, transaction history, saved beneficiaries, fund transfers, loan records, EMI payments, email verification, and account-balance audit information. Administrative pages support management activities, while customer-facing pages provide access to permitted banking operations. Only functions present in the final running version should be described as implemented.

2.2 Functional Requirements

Area	Requirement
Authentication	Accept valid credentials, reject invalid credentials, and apply required email verification/OTP flow.
Customer/account management	Store and display customer and account details; support permitted status and management actions.
Transfers and transactions	Validate transfer inputs, record transfer details, and maintain transaction history consistently.
Beneficiaries	Save and manage beneficiary records according to implemented status and deletion rules.
Loans and EMI	Store loan information and associated scheduled/payment records.
Reports	Retrieve transaction histories and aggregate summaries using SQL.

2.3 Technology Stack

- Front end: HTML, CSS, and JavaScript.
- Backend: Node.js with Express.
- Database: MySQL, using the database name fintech_banking_db in the development setup.
- Supporting components: authentication routes, email delivery for verification workflows, and scheduled EMI-related logic where enabled.

2.4 Non-functional Considerations

Important considerations include input validation, authorization on protected endpoints, consistent transaction handling, usable feedback for errors, responsive layout, secure storage of credentials, and protection of customer data. Security should be assessed through actual implementation and testing rather than inferred from the presence of a feature.

3. System Architecture and Workflow

3.1 Architecture Overview

The system follows a common web application architecture. A user interacts with the browser-based interface. JavaScript sends requests to the Express API. The backend validates the request, applies authentication and authorization rules, runs database operations through the database connection module, and returns a response to the interface. MySQL stores the persistent banking records.

3.2 Main Components

Front end: Provides customer and administrator pages, forms, status indicators, tables, and operation feedback. The existing IBI Bank branding and interface should be preserved in the submitted application.

Backend: Provides API routes for authentication and banking operations. Server-side validation and authorization are essential because front-end checks alone can be bypassed.

Database: Maintains customer, account, transaction, transfer, beneficiary, loan, EMI, and supporting records. Database objects provide reusable query logic and integrity checks.

3.3 Example Transfer Workflow

1. The customer selects an allowed recipient and enters an amount.
2. The interface submits the request to the backend.
3. The backend validates identity, account ownership, amount, recipient, and other applicable business rules.
4. The server performs the transfer-related database operations.
5. The transfer and corresponding transaction records are saved, and account balances are updated according to the implementation.
6. The interface displays the result and, where implemented, a transfer reference or receipt.

3.4 Authentication and Roles

The project includes user authentication workflows and role information. The report should describe the actual customer and administrator login flows, email verification or OTP behavior, password storage approach, and authorization middleware. Do not include passwords, OTPs, JWT secrets, SMTP credentials, or environment variable values in the report.

4. Database Design and Relational Schema

4.1 Main Tables

Table	Purpose
customer	Customer profile and contact details.
users	Login identity, role, and authentication-related fields.
account	Account number, customer association, account type, branch, balance, and status.
account_type / branch	Reference information for account classification and branch details.
transaction	Individual transaction history associated with accounts.
fund_transfer	Transfer sender, receiver, amount, status, date, and reference.
beneficiary	Saved beneficiary details and status.
loan_type / loan	Loan category information and customer loan records.
emi_payment	Scheduled and recorded EMI payment information.
account_balance_audit	Audit records related to account balance changes.
email_verification_codes	Email verification and OTP-related workflow data.

4.2 Key Relationships

The logical design associates accounts with customers, branches, and account types. Transactions are associated with accounts. Transfers identify sender and receiver accounts and may reference a saved beneficiary. Loans are associated with customers, and EMI records are associated with loans. Authentication identities are related to customer profiles through the application's registration/login design.

4.3 Keys and Constraints

Primary keys identify individual records. Foreign keys, where declared, enforce referential integrity between related tables. Unique constraints may be used for identifiers such as account numbers and login email addresses. The exact primary keys, foreign keys, column data types, indexes, and unique constraints must be taken from the actual MySQL schema export.

5. Normalization and Integrity

5.1 First Normal Form (1NF)

First Normal Form requires atomic field values and the absence of repeating groups. In this system, a customer is represented by a customer row, an account by an account row, a transaction by a transaction row, and each EMI record by an EMI row. This is preferable to storing multiple accounts or transactions as a list in a single field.

5.2 Second Normal Form (2NF)

Second Normal Form requires 1NF and the absence of partial dependencies on part of a candidate key. The tables use individual identifiers such as account_id, transaction_id, loan_id, and emi_id. Account details are stored in account, while individual transaction details are stored in transaction. EMI details are separated from the main loan record. For any composite-key table, the final schema should be examined for partial dependencies.

5.3 Third Normal Form (3NF)

Third Normal Form requires 2NF and no transitive dependency of non-key attributes on a candidate key. Branch details and account-type rules are stored in their own tables rather than duplicated as full descriptive records in each account. Loan records and EMI payment records are separated to support multiple payments per loan.

A schema detail deserves explicit review: loan_type contains category/rate information, while loan also has a loan_type field and an applied interest_rate. Confirm whether loan references the lookup table through an identifier or stores the category as text. If standard values are duplicated, document the reason or consider a future normalization improvement; do not modify the working schema just to make this report look consistent.

5.4 Integrity and Audit

Application validation and database constraints can help reject invalid values. The project's transaction-amount validation trigger is intended to reject invalid transaction amounts. The account-balance audit trigger is intended to record account balance changes. Their exact definitions and observed behavior should be included from the database-object inventory and verified against the actual schema.

5.5 Security and Privacy

Authentication data and banking records require careful handling. Public submissions should exclude secrets and should use synthetic or appropriately anonymized example data. Access control should be enforced on backend routes, not only by hiding interface elements. Error responses should avoid revealing credentials or sensitive internal information.

6. SQL Implementation and Advanced Database Features

6.1 Query Categories

The project's SQL query collection is organized by topic so that each requirement can be demonstrated independently.

File	Demonstration
01_basic_select_queries.sql	Basic retrieval and filtering.
02_aggregate_group_by_queries.sql	COUNT, SUM, AVG, and grouped summaries.
03_join_queries.sql	Related records across multiple tables.
04_subqueries.sql	Queries using nested SELECT statements.
05_correlated_subqueries.sql	Subqueries linked to the outer query.
06_dashboard_report_queries.sql	Summary information for dashboard/report use.
07_transaction_demo.sql	Transaction control demonstration using a safe demo object.
08_advanced_sql_inventory.sql	Inspection of existing views, routines, triggers, and indexes.

6.2 Views

The reported views are vw_account_details, vw_fund_transfer_details, vw_loan_emi_summary, and vw_transaction_history. Views can encapsulate joins or selected columns and provide reusable query interfaces. Include their actual CREATE VIEW definitions in the final database export or supplementary SQL evidence.

6.3 Stored Procedures and Function

The reported stored procedures are sp_account_transaction_history and sp_customer_account_summary. The reported function is fn_calculate_monthly_interest. The report should state their input parameters, output/result behavior, and purpose based on their actual definitions.

6.4 Triggers and Indexes

The reported triggers are trg_account_balance_audit and trg_validate_transaction_amount. Indexes support lookup and join performance, while triggers run automatically in response to defined database events. Include actual SHOW CREATE and SHOW INDEX outputs as evidence; the inventory script lists objects but is not itself a rebuild script.

6.5 Transaction Control

The transaction demonstration should explain the purpose of START TRANSACTION, COMMIT, and ROLLBACK. A demonstration using a temporary or isolated test table is safer than testing by modifying real account balances. Include the actual statements and observed output.

7. Testing Plan and Results

7.1 Testing Approach

Testing should cover functional behavior, validation, access control, database integrity, and query execution. Each test should record its input, expected result, actual result, pass/fail status, and evidence reference. This document is a report starter: the table below must be completed from actual testing, and no test should be marked passed without evidence.

ID	Test	Expected outcome	Result
T01	Start backend	Server starts and MySQL connects.	[Record]
T02	Invalid login	Request is rejected safely.	[Record]
T03	Role protection	Unauthorized role is denied.	[Record]
T04	Valid transfer	Transfer records and balances remain consistent.	[Record]
T05	Invalid amount	Invalid request is rejected.	[Record]
T06	Transaction history	Relevant records are returned.	[Record]
T07	SQL joins/reports	Queries return appropriate results.	[Record]
T08	Stored routines/triggers	Database logic behaves as defined.	[Record]
T09	Rollback demonstration	Uncommitted demo changes are rolled back.	[Record]

7.2 Evidence

Attach genuine screenshots of the running interface and SQL output. Suggested evidence includes login/verification, customer and account management, transfer/receipt, loan/EMI pages, representative SELECT/JOIN/aggregate outputs, and database-object definitions. Do not invent screenshots or results.

7.3 Limitations

Record known limitations found during implementation or testing. Potential future work may include automated integration tests, enhanced audit reports, improved deployment/backup procedures, and additional reporting filters. Only identify limitations that apply to the submitted version.

8. Conclusion, Future Scope, and References

8.1 Conclusion

The IBI Bank Banking and Digital Transaction Management System combines a web interface, backend APIs, and a relational MySQL database to represent common banking information and workflows. Its data model separates core entities such as customers, accounts, transactions, transfers, beneficiaries, loans, and EMI payments. The associated SQL deliverables demonstrate a range of database concepts, including retrieval, aggregation, joins, subqueries, reusable views, stored routines, triggers, indexes, and transaction control.

The strength of the final submission depends on aligning this report with the actual implementation: the final schema, SQL object definitions, application screenshots, and recorded test results should all refer to the same version of the project.

8.2 Future Scope

Possible future improvements include broader automated test coverage, stronger audit and monitoring dashboards, deployment hardening, scheduled backups and recovery exercises, expanded reporting filters, and continued review of access control and data privacy. These items should be presented as future work unless they are already implemented.

8.3 References and Supporting Material

1. BTY39105 DBMS PBL university proforma and course instructions supplied for the project.
2. MySQL documentation: https://dev.mysql.com/doc/
3. Node.js documentation: https://nodejs.org/docs/
4. Express documentation: https://expressjs.com/
5. Project SQL scripts, database schema export, and source code maintained by the project team.